



UnB - FCTE



conhecimento em movimento
sociedade em transformação

Estruturas de Dados e Algoritmos 1

Aula 06

Ponteiros II

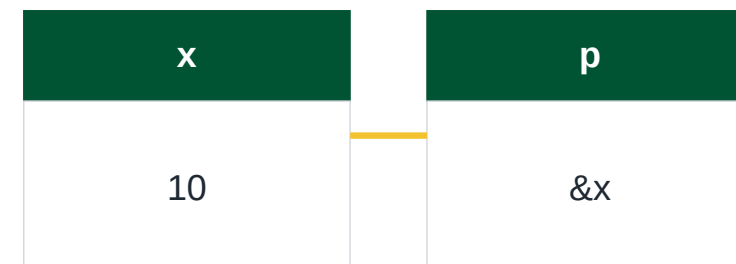
parâmetros - modificação indireta - arrays e ponteiros



**passar o endereço permite
modificar o dado do chamador**

Objetivos da aula

- Explicar por que a passagem comum de parâmetros não altera a variável do chamador.
- Usar ponteiros como parâmetros para permitir modificação indireta.
- Rastrear funções que recebem endereços.
- Relacionar parâmetros de array com ponteiros.
- Usar índices e indireção para acessar elementos de um vetor.
- Preparar a implementação da PC2.

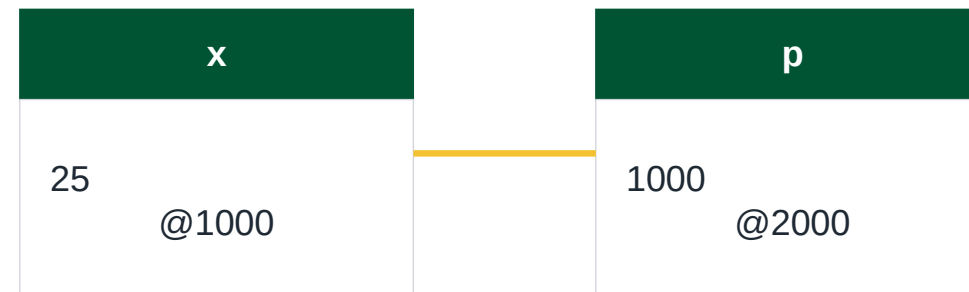


Ideia central:
passar o endereço, não uma cópia do valor.

Retomada: p, &x e *p

```
int x = 10;  
int *p = &x;  
  
*p = 25;
```

p armazena um endereço.
***p** acessa o valor naquele endereço.



**Hoje vamos transportar essa ideia
para dentro de funções.**

Por que uma função comum não altera x?

```
void zerar(int a)
{
    a = 0;
}

int main(void)
{
    int x = 10;
    zerar(x);
}
```

main: x	
10	@1000

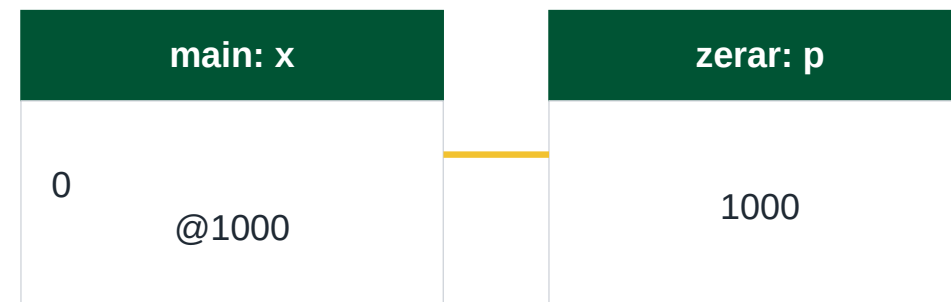
zerar: a	
10	@3000

**a recebe uma cópia do valor de x.
Alterar a não altera x.**

Passando o endereço

```
void zerar(int *p)
{
    *p = 0;
}

int main(void)
{
    int x = 10;
    zerar(&x);
}
```



p recebe o endereço de x.
***p modifica diretamente x.**

Valor x endereço: compare as chamadas

Passagem por valor

```
void f(int a);  
...  
f(x);
```

A função recebe uma cópia.
Não há acesso direto a x.

Passagem de endereço

```
void g(int *p);  
...  
g(&x);
```

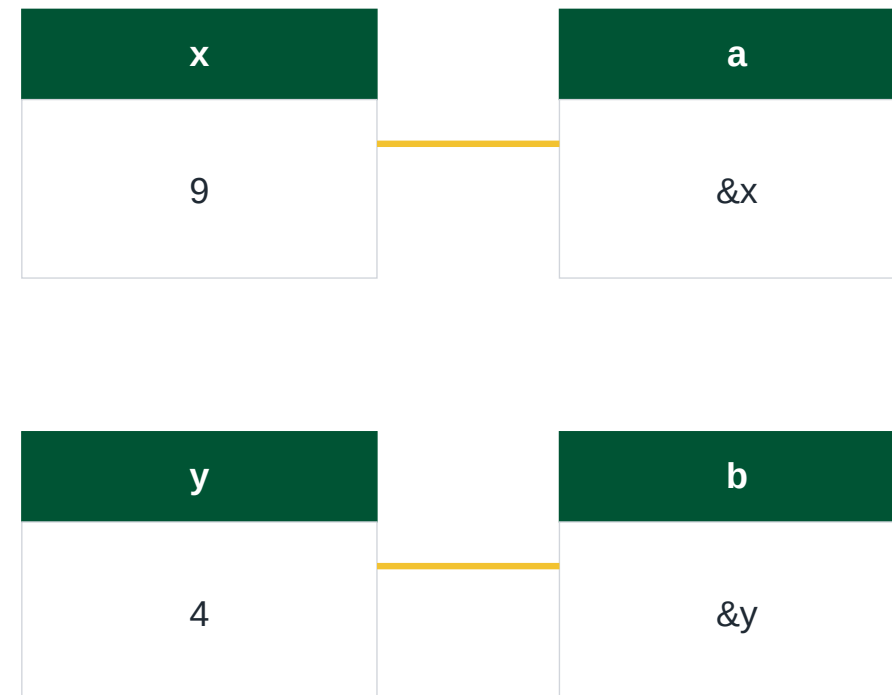
A função recebe o endereço.
*p pode alterar x.

Em C, os parâmetros continuam sendo passados por valor; aqui o valor copiado é um endereço.

Exemplo clássico: troca de duas variáveis

```
void trocar(int *a, int *b)
{
    int aux = *a;
    *a = *b;
    *b = aux;
}

int x = 4, y = 9;
    trocar(&x, &y);
```



O aux guarda um valor, não um endereço.

Rastreamento de uma função com ponteiro

```
void ajustar(int *p, int d)
{
    *p = *p + d;
    d = 0;
}

int x = 7;
int k = 3;

    ajustar(&x, k);
```

Perguntas

- Qual o valor final de x?
- Qual o valor final de k?
- Por que d=0 não altera k?
- Por que *p altera x?

Arrays e ponteiros: a relação

```
int v[4] = {10, 20, 30, 40};
```

```
v[0]      /* 10 */  
          *v      /* 10 */
```

v[0]	v[1]	v[2]	v[3]
10	20	30	40

Em muitas expressões, v representa o endereço do primeiro elemento do vetor.

Por isso, $*v$ acessa o primeiro elemento.

Índices e indireção

$v[i]$

$*(v + i)$

As duas formas acessam o mesmo elemento.

$v[0]$	$v[1]$	$v[2]$	$v[3]$
10	20	30	40

**$v + i$ avança i elementos,
não i bytes.**

Hoje basta compreender a equivalência;
não precisamos explorar aritmética avançada.

Parâmetro de array em uma função

```
void somarUm(int v[], int n)
{
    int i;

    for (i = 0; i < n; i++)
        v[i] = v[i] + 1;
}
```

O que acontece?

- a função recebe acesso aos elementos do vetor;
- alterar `v[i]` altera o vetor do chamador;
- o tamanho `n` precisa ser informado separadamente.

Para parâmetros, `int v[]` e `int *v` expressam a mesma ideia de acesso ao vetor.

Duas formas de escrever o parâmetro

```
void f(int v[], int n);
```

```
void f(int *v, int n);
```

Em uma lista de parâmetros de função, ambas permitem percorrer e modificar os elementos do vetor.

Prefira a notação que torne a intenção mais clara para quem lê o código.

Miniatividade: sem compilador

```
void f(int *x, int v[])
{
    *x = v[0];
    v[1] = *x + v[2];
}

int a = 5;
int v[3] = {2, 4, 6};

        f(&a, v);
```

Determine:

- valor final de a;
- conteúdo final de v[0];
- conteúdo final de v[1];
- conteúdo final de v[2].

Use um diagrama para justificar.

WS02 - Continuação: Ponteiros II

Nesta segunda parte do WS02:

- compare passagem de valor e de endereço;
- rastreie funções com parâmetros ponteiro;
- complete uma função de troca;
- relacione arrays e ponteiros;
- prepare a solução da PC2.



Pergunta-chave:

**o que precisa ser passado para uma função
alterar um dado do chamador?**

PC2 - Funções com ponteiros

Implementação extraclasse

1. implementar `trocar(int *a, int *b);`
2. implementar `extremos(int v[], int n, int *menor, int *maior);`
3. compilar e testar com casos fornecidos e próprios.

```
void trocar(int *a, int *b);  
  
void extremos(int v[], int n,  
              int *menor, int  
              *maior);
```

**A prática transforma o raciocínio do WS02
em código ANSI C executável.**

Fechamento

Síntese

- Funções comuns recebem cópias dos argumentos.
- Um endereço também pode ser passado como valor.
- Com `int *p`, a função pode alterar o dado apontado.
- Arrays permitem acesso aos elementos a partir de seu endereço inicial.
- Em parâmetros, `int v[]` e `int *v` são equivalentes.

Próxima aula

**Structs II + Ponteiros:
ponteiros para estruturas,
operador `->` e modificação de
structs.**

Leve para a próxima aula as dúvidas encontradas na PC2.